

Xiaomi Boot Logo Replacement: Cross-Device Research Findings

1. Your Key Observation Was Correct

In your README you wrote:

"The device will not be bricked by flashing a borked image, but instead display default Qualcomm boot screen if you try to repack with UEFITool (we have determined that probably some other tool needs to be used for a successful repack)."

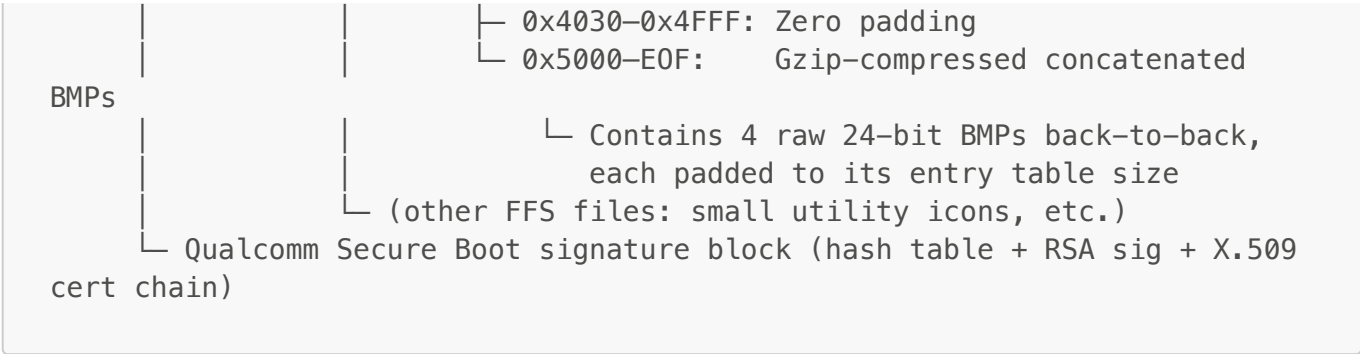
This is the critical clue. The Qualcomm default boot screen means **ABL loaded and ran, but couldn't parse the logo data** — it fell back to a hardcoded default. If the signature check had failed, the device would have refused to boot entirely (or shown a secure boot failure), not fallen back gracefully to a default splash. This strongly suggests that **signature verification on imagefv is bypassed when the bootloader is unlocked**, exactly as it is for `boot.img` and other AVB-protected partitions.

The problem was never signing. It was **structural corruption introduced by UEFITool during reinsertion**.

2. The imagefv Structure (Full Layer Breakdown)

We fully mapped the `imagefv` partition structure through manual binary analysis. Here's every layer:

```
imagefv.img (raw partition)
├─ ELF container (Qualcomm MBN format with signature appendix)
│   ├── ELF Program Header → points to UEFI Firmware Volume payload
│   └─ Payload: UEFI Firmware Volume (FV)
│       ├── FV Header (signature "_FVH", 0x48 bytes, with length +
checksum)
│       │   ├── Block Map entries
│       │   └─ LZMA-compressed inner FV
│       │       └─ Inner UEFI Firmware Volume
│       │           ├── FV Header
│       │           └─ FFS File: "logo.img" (GUID: B90FFA41-D22C-4B0E-AC8B-
65B98ACE057D)
│               ├── FFS Header (name, size, type, state, checksums)
│               └─ Raw Section
│                   ├── 0x0000-0x3FFF: Zero padding (16 KiB)
│                   ├── 0x4000-0x400F: LOGO!!!! header
│                   │   ├── bytes 0-7: Magic "LOGO!!!!"
│                   │   └─ bytes 8-11: Unknown field (0x14 on odin,
0x19 on nabu)
│                   └─ bytes 12-15: Unknown field (0x13F11 on
odin, 0x18275 on nabu)
│                       ├── 0x4010-0x402F: Entry table (4 entries × 8
bytes)
│                       └─ Each entry: [uint32 offset] [uint32
padded_size]
```



3. Why UEFITool Breaks It

When you extract the `logo.img` FFS body with UEFITool and reinsert it, UEFITool must:

- 1. Rebuild the inner UEFI FV (recalculate FFS headers, section sizes, FV checksums)
- 2. Re-compress with LZMA
- 3. Rebuild the outer FV (update size, block map, checksum)
- 4. Re-wrap in the ELF container

Any of these steps can introduce subtle corruption:

- **LZMA recompression** may produce different-sized output, causing the outer FV to change size. If the new FV doesn't match the original partition size exactly, the ELF program header offsets for the signature block shift, and the whole structure becomes unparseable to ABL.
- **FV/FFS header recalculation** — UEFITool may not correctly handle Qualcomm's specific FV layout conventions (e.g., the block map, alignment requirements).
- **Outer FV size mismatch** — If UEFITool shrinks or expands the outer FV, ABL can't find what it expects at the expected offsets.

Our tool avoids all of this by working at the byte level: we decompress, modify only the logo payload, recompress, and then surgically patch the bytes back in while preserving the exact original outer FV size (padding with `0xFF` if the new compressed data is smaller).

4. The Hardcoded Slot Size Problem (nabu vs. other devices)

We tested your `decompress.py` and `compress.py` directly against our MIX 4 `logo.img` data. Here's what we found:

Entry Table Comparison

Field	Xiaomi Pad 5 (nabu)	Xiaomi MIX 4 (odin)
Screen resolution	1600×2560	1080×2400
Unknown field 1	0x19 (25)	0x14 (20)
Unknown field 2	0x18275	0x13F11
Logo 0 padded size	0xBB8038 (12,288,056)	0x76A738 (7,776,056)
Logo 1 padded size	0xBB8036 (12,288,054)	0x73AFD6 (7,581,654)
Logo 2 padded size	0xBB8038 (12,288,056)	0x76A738 (7,776,056)

Field	Xiaomi Pad 5 (nabu)	Xiaomi MIX 4 (odin)
Logo 3 padded size	0xBB8038 (12,288,056)	0x76A738 (7,776,056)

The padded sizes are **device-specific** — they correspond to the 24-bit uncompressed BMP size for each device's screen resolution plus the 54-byte BMP header. Your tool hardcodes 0xBB8038 which is correct for nabu's 1600x2560 display, but when run against our 1080x2400 MIX 4 image:

- **Logo 0:** Extracted correctly (both start at offset 0)
- **Logo 1:** Corrupted — nabu offset 0xBB8038 lands in null padding; real offset is 0x76A738
- **Logo 2:** Corrupted — nabu offset 0x177006E vs real 0xEA570E
- **Logo 3:** Empty — nabu offset 0x23280A6 is past the end of decompressed data

The compress.py output was 20,969 bytes larger than the original and contained the nabu entry table, making it doubly wrong for MIX 4.

Fix

The entry table at 0x4010 already contains the correct offsets and sizes for any device. The tool just needs to **read them dynamically** instead of using hardcoded values. This would make your tool universal across all Xiaomi devices using the LOGO!!!! format.

5. What We Built

We developed logo_tool.py — a single Python script that handles the complete round-trip without UEFITool:

```
python3 logo_tool.py extract imagefv.img logos/      # Extract all 4 boot logos
python3 logo_tool.py repack  imagefv.img logos/ out.img # Repack with custom logos
python3 logo_tool.py info    imagefv.img             # Show partition structure
```

Key design decisions:

- **Reads all sizes/offsets from the binary** — no hardcoded values, works on any device
- **Preserves original outer FV size** — pads with 0xFF to maintain exact partition geometry
- **Recalculates all UEFI checksums** — FV header checksum, FFS header checksum, FFS data checksum
- **Handles ELF wrapper natively** — no need for UEFITool at all
- **BMP validation** — ensures replacement images match expected dimensions and bit depth
- **PNG/BMP input** — accepts PNG files and converts to 24-bit BMP automatically

6. Current Status — Not Yet Validated on Device

Important disclosure: As of this writing, we have **not yet flashed a custom imagefv image** to our MIX 4. All findings so far are based on binary analysis, round-trip integrity testing (extract → repack → byte

comparison), and cross-device format validation against your nabu data. The tool produces output that passes our structural checks — correct UEFI checksums, matching FV sizes, valid LOGO!!!! entry tables — but we have not confirmed that ABL actually accepts and renders a repacked image on a live device.

Ideally, we'd love to collaborate and validate the tool on your Pad 5 first — since you've already done flashing experiments on nabu, it would be a much safer starting point than jumping straight to an untested platform like the MIX 4 (odin). If you've done any further experiments since your last README update, we'd be very interested to hear the outcome.

7. Open Questions

1. **Has anyone actually confirmed a successful custom logo flash on any Xiaomi UEFI device?** Your README mentions restoring an older official `imagefv` worked — that's promising since it proves the flash path works.
2. **The two unknown header fields** (`0x14/0x19` and `0x13F11/0x18275`) — do you have any theories? They might be version numbers, total decompressed size indicators, or BMP count/flags. Understanding them would help ensure repacked images are fully correct.
3. **Would you be interested in testing on your Pad 5?** We'd love to validate the tool on nabu as well. The code should work out of the box since it reads everything dynamically.

8. Repository

The tool and all research artifacts are available at: *(will share link)*

The toolkit includes:

- `logo_tool.py` — Extract, repack, and inspect boot logos from `imagefv` partitions
- `dump_logo.sh` — One-command ADB helper that dumps `imagefv` from any connected Xiaomi device, detects A/B slots, and feeds the image directly into `logo_tool.py` for extraction

Happy to collaborate on this in whatever form works — GitHub issues, a shared repo, email, etc. Your initial reverse engineering of the `LOGO!!!!` format saved us significant time, and I think together we can turn this into a reliable, multi-device solution.